

---

# 多変量解析及び大規模計算論

第10回：大規模計算論パート  
並列・分散コンピューティングを理解する

教員：玉森，内種

---

# 目次

---

- 並列コンピューティング
  - コンパイラレベル(SIMD, パイプライン)
  - マルチコアレベル(マルチスレッド, OpenMP)
  - 計算クラスタレベル(MPI, HADOOP)
- [予習]モンテカルロ積分(次回の内容)

---

# 並列コンピューティング

---

- ソフトウェアで並列処理を書く
  - コンパイラレベル
    - CPUの機能を最大限に利用
  - マルチコアレベル
    - 複数のCPUの連携を指示
    - メモリー・ディスクは共用
  - 計算クラスタレベル
    - 複数の計算機の連携を指示
    - メモリー・ディスクは分散

---

# コンパイラレベル並列化

---

- SIMD
  - Single Instruction/Multiple Data
  - 1命令で足し算512bit(64bitを8セット)
- パイプライン
  - 複数のステージから成る処理をパイプライン化し各ステージを並行して実行
- 役に立つか？
  - 同じ計算, 依存関係の内計算では役立つが・・・
  - コンパイラレベル => はまると何倍も効果的(c/c++)
  - 非コンパイラレベル = 応用範囲が広い(pythonモジュール)

---

# マルチコアレベル並列化

---

- スレッド並列化
  - スレッド=特定の処理を行うための命令の流れ
  - プロセスは1つ以上のスレッドを含む
    - 例：調理(4人分)→食事x4回→片付け(4人分)
- OpenMP
  - 共有メモリ・マルチスレッド型並列計算
  - 標準化されたAPIを提供
  - コア数指定・共有変数の管理

---

# 計算クラスタレベル

---

- MPI
  - Message Passing Interface
  - データの送受信・命令待ちを制御
  - 共通データもプログラマが送る手続きをする
- HADOOP
  - Javaのみ
  - データの送受信・命令待ちを制御
  - 共通データは裏でミドルウェア任せ

---

# (脱線) ハードウェア並列化

---

- クラウドコンピューティング
  - 計算機をネットワークでつなげて計算機クラスターとして構築・設定するのは大変
  - スパコンは性能が出るが、上記のコストが高い
- 仮想環境で作る計算クラスターが登場
  - Kubernetes：仮想環境のデプロイ・スケーリング・管理
    - Docker-compose：仮想環境の組み合わせ
    - Docker：ライトウェイト仮想化

---

# この講義の演習では

---

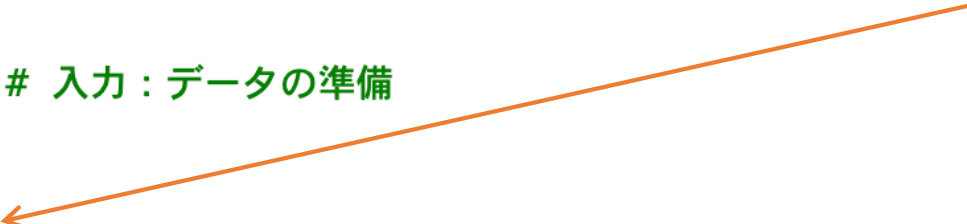
- コンパイラレベル
  - C / C++ など (この講義では扱わない)
- マルチコアレベル
  - スレッド並列 (pythonで扱える[次回の演習へ])
- 計算クラスタレベル
  - 計算機の準備が必要 (この講義では扱わない)



# スレッド並列の例(演習は次回)

- Pythonのmultiprocessingモジュール

```
30 def merge_sort_parallel(data):
31     ... if len(data) <= 1:
32     ...     return data
33
34     .....
35     ... 演習：データをn分割してnプロセス同時に実行するプログラムに変更せよ
36     .....
37     ... mid = len(data) // 2 # 入力：データの準備
38
39     ... # 計算：2並列で実行
40     ... with multiprocessing.Pool(2) as pool:
41     ...     left, right = pool.map(merge_sort, [data[:mid], data[mid:]])
42
43     ... # 出力：結果のマージ
44     ... return merge(left, right)
45
```



---

## [予習] モンテカルロ積分

---

- 何を並列に計算するか？
  - 確率分布を実現値で近似
  - 確率分布の例：時系列  $x_t \sim p(x_t | x_{t-1})$
- モンテカルロ積分 [Monte Carlo integration]  
(wikipediaより)
  - 擬似乱数を用いた数値積分の手法で、これはモンテカルロ法の一つであり、  
定積分を数値的に計算する方法

---

## [予習] ベイズの定理

---

- ベイズの定理
  - $p(\theta_i|D) = \frac{p(D|\theta_i)p(\theta_i)}{\int p(D|\theta_i)p(\theta_i)d\theta_i}$
  - 分母  $\int p(D|\theta)p(\theta)d\theta$  が定積分
- 導出方法は別資料
  - 03-01\_条件付き確率・確率の乗法定理.pdf
  - 03-02\_ベイズの定理.pdf
  - 03-03\_事象の独立.pdf

## [予習] モンテカルロ積分

- ベイズの定理

- $p(\theta_i|D) = \frac{p(D|\theta_i)p(\theta_i)}{\int p(D|\theta)p(\theta)d\theta}$
- 分母  $\int p(D|\theta)p(\theta)d\theta$  が定積分

- 確率分布の粒子近似

- $E[f(\theta)] \approx \frac{1}{N} \sum_{i=1}^N f(\theta_i) = \frac{1}{N} \sum_{i=1}^N p(D|\theta_i)$
- 粒子  $i$  はランダムサンプルなので出現確率  $\frac{1}{N}$
- 尤度関数（評価関数）  $f(\theta_i)$  を合計（分母とする）

## [予習] モンテカルロ積分

- ベイズの定理

- $p(\theta_i|D) = \frac{p(D|\theta_i)p(\theta_i)}{\int p(D|\theta)p(\theta)d\theta}$

- 分母  $\int p(D|\theta)p(\theta)d\theta$  が定積分

- 確率分布の粒子近似

- $E[f(\theta)] \approx \frac{1}{N} \sum_{i=1}^N f(\theta_i) = p(\theta_i) \sum_{i=1}^N p(D|\theta_i)$

- 粒子  $i$  はランダムサンプルなので出現確率  $\frac{1}{N}$

- 尤度関数（評価関数）  $f(\theta_i)$  を合計（分母とする）

# [予習] モンテカルロ積分

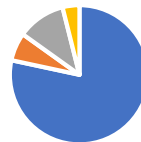
- 尤度の計算
  - 進化計算では評価関数（与えられる）
  - ベイズ最適化では評価モデル（学習される）
  - 粒子フィルタでは観測関数（与えられる）
- 予測(ベイズの定理計算)の結果イメージ

評価関数（大=良）



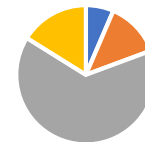
■ 粒子1: $f(\theta)=0.1$  ■ 粒子2: $f(\theta)=0.5$   
■ 粒子3: $f(\theta)=1.2$  ■ 粒子4: $f(\theta)=3.3$

損失関数（小=良）



■ 粒子1: $f(\theta)=0.01$  ■ 粒子2: $f(\theta)=0.12$   
■ 粒子3: $f(\theta)=0.07$  ■ 粒子4: $f(\theta)=0.2$

観測関数（小=良）



■ 粒子1: $f(\theta)=10$  ■ 粒子2: $f(\theta)=5$   
■ 粒子3: $f(\theta)=1$  ■ 粒子4: $f(\theta)=4$

---

# 要するに何がしたい？

---

- 最初に（ランダムな）分布にしたがって粒子をばらまく
- 粒子の評価が他の粒子に比べてどれぐらい良いかを計算
- [次々回]粒子の良さ（ルーレット）に従って確率的にリサンプリング
- [次々回]リサンプリングした新しい確率分布に従ってまた粒子をばらまく→以降繰り返し

---

# 第10回まとめ

---

- 並列コンピューティング
  - コンパイラレベル(SIMD, パイプライン)
  - マルチコアレベル(スレッド, OpenMP)
  - 計算クラスタレベル(MPI, HADOOP)
- (脱線) 分散コンピューティング
- [予習]モンテカルロ積分